

from Beginner to Master

DevOps를 위한 **Docker** 가상화

Section 4. Building and Managing Containerized Applications

```
book:
  def setData(self, title, price, author):
    self.title = title
    self.price = price
    self.author = author

var fs = require('fs');
fs.readFile('./ONE.txt', /* 1 */,
  function (err, data) {
    console.log(data); // 3
  });
ApplicationDelegate {
```

```
public static void main(String[] args)

<servlet>
  <servlet-name>BoardController</servlet-name>
  <servlet-class>com.joneconsulting.controller.BoardCont
  <init-param>
    <param-name>user_name</param-name>
    <param-value>Kenneth Lee</param-value>
  </init-param>
</servlet>
```

Winterface

Dowon Lee

수강생 19K 강의평점 4.8(1K)



멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

수정하기

강의 전체 6



[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

▶ 학습중

★ 0강 / 25강 (0%)

818명



멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

▶ 학습중

★ 0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

▶ 학습중

★ 81강 / 83강 (98%)

독점 2709명



Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

▶ 학습중

★ 156강 / 156강 (100%)

독점 5531명



Spring Boot를 이용한 RESTful Web Services 개발

▶ 학습중

★ 3강 / 48강 (6%)

독점 3862명



[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

▶ 학습중

★ 14강 / 14강 (100%)

4813명

이 되었던 때가 있었습니
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구
사람입니다. 개발자로서, 강
지만, 그래도, 남들보다 조

ecture, Blockchain,

- Section 0: Introduction to Cloud Native Technologies
- Section 1: Docker Essentials - Container
- Section 2: Docker Essentials - Image
- Section 3: Docker Network and Storage
- **Section 4: Building and Managing Containerized Application**
- Section 5: Container Orchestration
- Section 6: Continuous Integration and Continuous Deployment
- Section 7: Docker Security
- Section 8: Logging and Monitoring
- Section 9: Advanced Docker Usage
- Section 10: Capstone Project

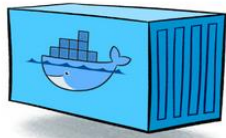
Section 4.

Building and Managing Containerized Application

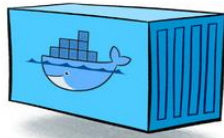
- Multi Container를 이용한 IT 서비스 구축
- Docker Compose
- Multi Container 관리
- Docker Compose를 활용한 IT 서비스 구축

멀티 컨테이너를 이용한 IT 서비스 구축

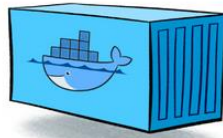
멀티 컨테이너로 구축하는 IT 서비스



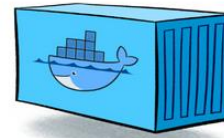
Kafka



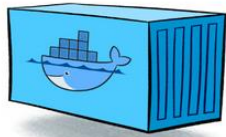
Zipkin



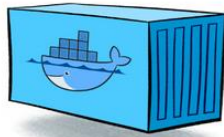
Prometheus



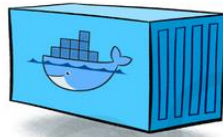
Grafana



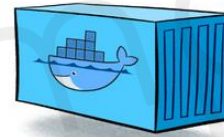
MariaDB



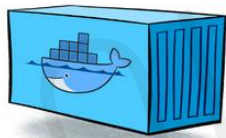
RabbitMQ



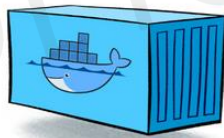
Fluentd



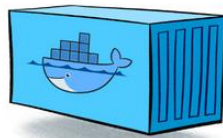
Elasticsearch



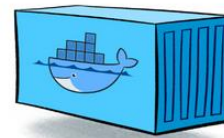
user-service



order-service



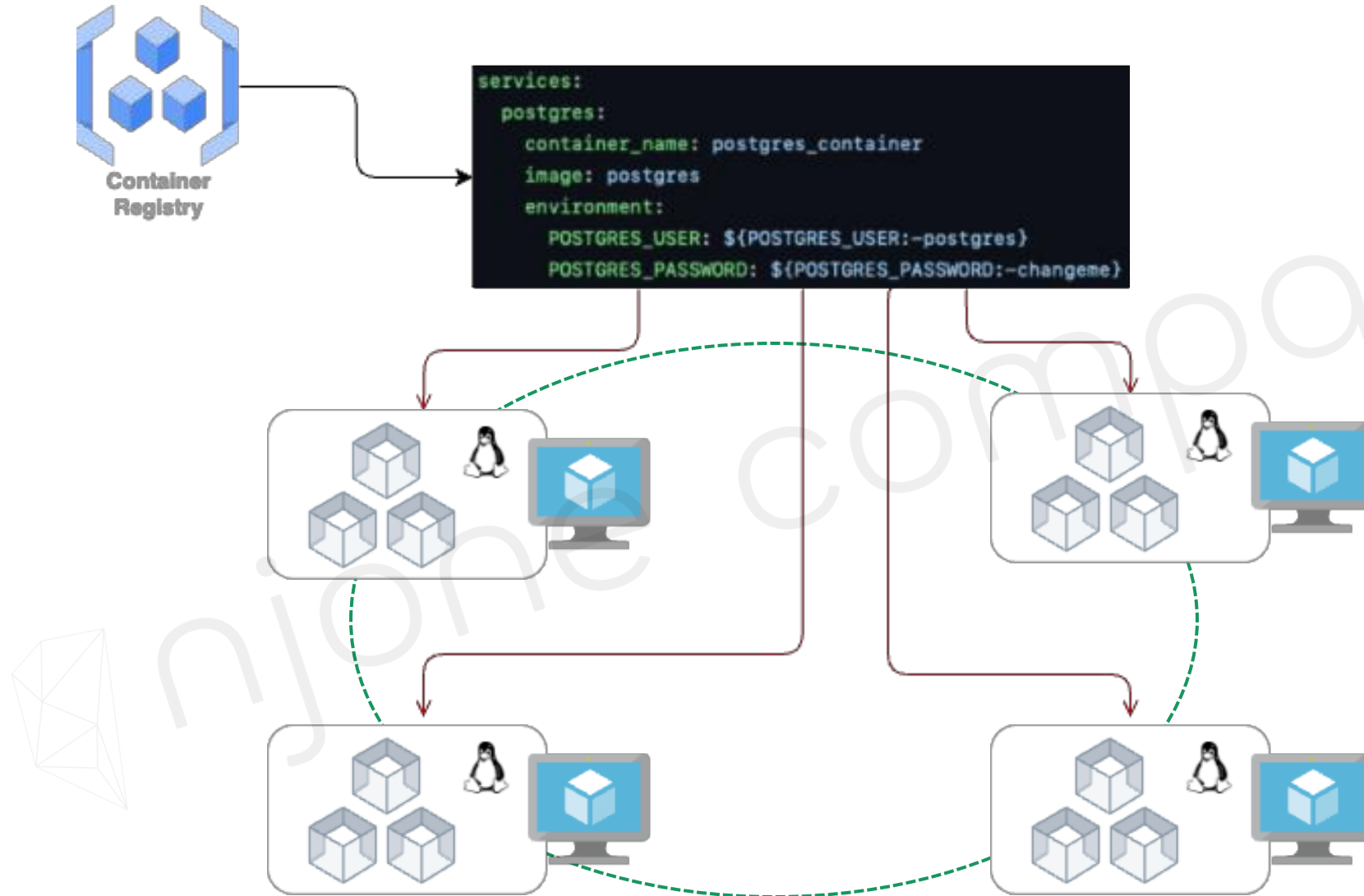
payment-service



delivery-service

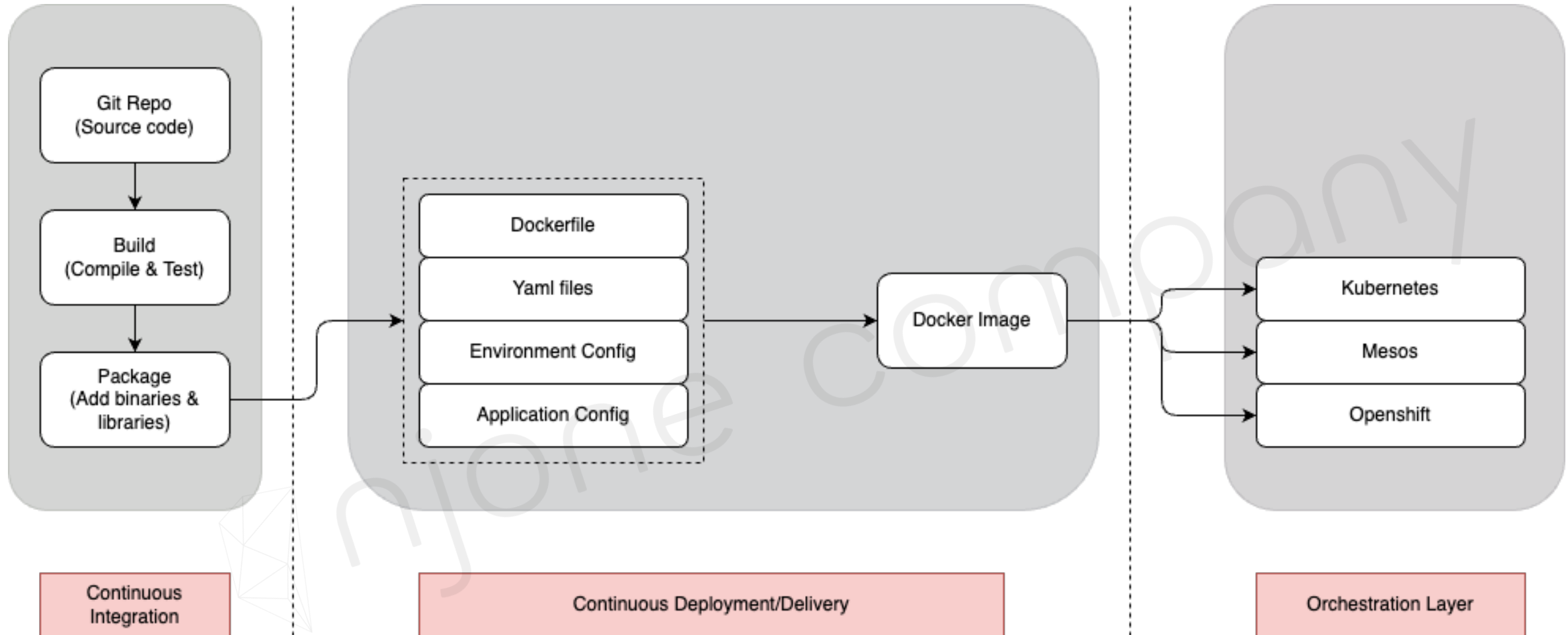
멀티 컨테이너를 이용한 IT 서비스 구축

멀티 컨테이너로 구축하는 IT 서비스



멀티 컨테이너를 이용한 IT 서비스 구축

멀티 컨테이너 환경 설계 및 배포



Docker Compose 개요

- Container 애플리케이션을 정의하고 실행하는 도구
- 한 번에 여러 개의 컨테이너 동시 실행 → 각 컨테이너별로 별도의 명령어 실행 가능
- 다른 컨테이너와의 접속을 쉽게 구성
 - | 복잡한 설정을 쉽게 관리하기 위한 도구
 - | Docker 생성, 설정 관련 된 작업을 작성해 놓은 Script 파일 사용 → Docker Compose 파일

```
flask-redis-redis-1 | 1:M 31 May 2023 09:44:33.692 * <ReJSON> version: 20011 git sha: 25ff494 branch: H
EAD
flask-redis-redis-1 | 1:M 31 May 2023 09:44:33.692 * <ReJSON> Exported RedisJSON_V1 API
flask-redis-redis-1 | 1:M 31 May 2023 09:44:33.692 * <ReJSON> Enabled diskless replication
flask-redis-web-1    | * Serving Flask app 'app'
flask-redis-web-1    | * Debug mode: on
flask-redis-web-1    | WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
flask-redis-web-1    | * Running on all addresses (0.0.0.0)
flask-redis-web-1    | * Running on http://127.0.0.1:8000
flask-redis-web-1    | * Running on http://172.21.0.3:8000
flask-redis-web-1    | Press CTRL+C to quit
```

Docker Compose 파일 작성

- docker-compose.yml 파일

```
version: '3.1'

service:
  servicename:
    image: # optional
    command: # optional
    environment: # optional
    volumes # optional
  servicename2: # if have second service...
volumes: # optional

network: # optional
```

- 실행

```
$ docker-compose up
```

```
$ docker-compose -f docker-compose1.yml up
```

Docker Compose를 활용한 Multi Container 관리

- docker-compose.yml 파일 작성 명령어

명령어	설명
image	Docker Image 지정
build	Dockerfile을 이용하여 Docker Image 빌드
command/entrypoint	컨테이너 안에서 작동하는 명령어 지정
ports/expose	컨테이너 간 통신을 위한 Port 설정
depends_on	서비스 의존 관계 정의 (절대적 X)
environment/env_file	컨테이너 환경변수 지정
container_name/labels	컨테이너 정보 설정
volumes/volumes_from	컨테이너 데이터 관리

Docker Compose 실행 명령어

- \$ docker-compose <OPTIONS>

Options	설명
up	컨테이너 생성/시작
ps	컨테이너 목록 표시
logs	컨테이너 로그 출력
run	컨테이너 실행
stop	컨테이너 정지
port	공개 포트 번호 표시
rm	컨테이너 삭제
config	구성 확인
down	컨테이너/리소스 삭제

Docker Compose로 DB 구축

- 실습) Mysql DB를 위한 Docker Compose 파일 작성

```
version: "3.1"
services:
  my-db:
    platform: linux/amd64 # for apple chip
    image: mariadb:latest
    environment:
      - MARIADB_ALLOW_EMPTY_ROOT_PASSWORD=true
      - MARIADB_DATABASE=mydb
    ports:
      - 3306:3306
    volumes:
      - /Users/edowon/Desktop/Work/14.online/devops-docker/docker-compose/data:/var/lib/mysql
```

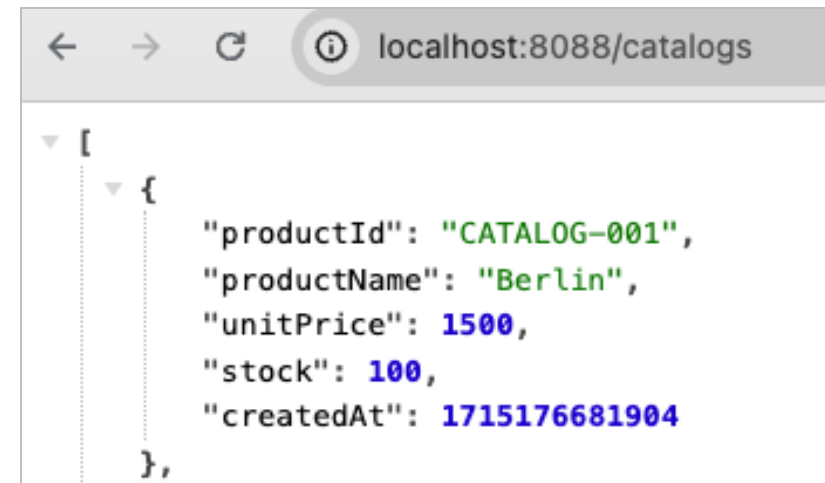
Docker Compose를 활용한 IT 서비스 구축

Docker Compose로 Backend 서비스 구축

- 실습) Backend 서비스를 위한 Docker Compose 파일 작성

```
version: "3.1"
services:
  my-backend:
    image: edowon0623/my-backend:1.0
    environment:
      - spring.datasource.url=jdbc:mariadb://my-db:3306/mydb
      - spring.datasource.password=
    ports:
      - 8088:8088
    depends_on:
      - my-db
    networks:
      - my-network
```

```
networks:
  my-network:
    external: true
```



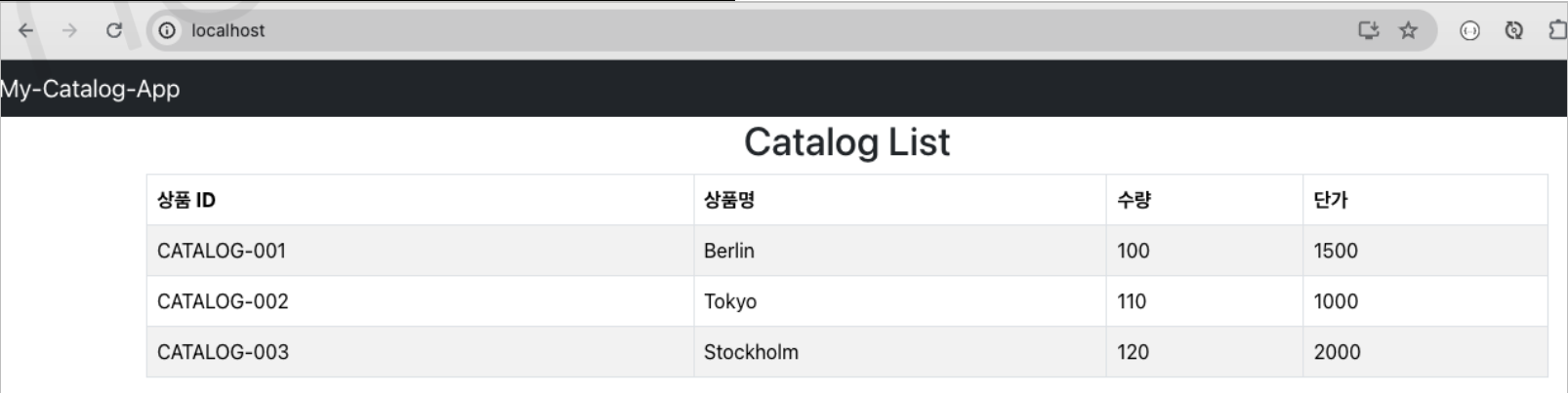
Docker Compose를 활용한 IT 서비스 구축

Docker Compose로 Frontend 서비스 구축

- 실습) Frontend 서비스를 위한 Docker Compose 파일 작성

```
version: "3.1"
services:
  my-frontend:
    image: edowon0623/my-frontend:1.0
    ports:
      - 80:80
    depends_on:
      - my-db
    networks:
      - my-network
```

```
networks:
  my-network:
    external: true
```



상품 ID	상품명	수량	단가
CATALOG-001	Berlin	100	1500
CATALOG-002	Tokyo	110	1000
CATALOG-003	Stockholm	120	2000